Verteilte Systeme — Übungsblatt 3: Synchronisation I

Prof. Dr. Oliver Haase, Dr. Thomas Zink

Aufgabe 1: Ring-Topologie

Momentan kommunizieren die Aqualife-Klienten sternförmig mit dem Broker, aber nicht direkt miteinander. Nicht nur die Registrierung und Deregistrierung, sondern auch die Übergabe von Fischen laufen über den Broker.

In dieser Aufgabe sollen Sie diese Sternstruktur auf eine Ringstruktur umstellen. Das bedeutet, dass Klienten Fische direkt miteinander austauschen, ohne hierfür den Broker zu verwenden. Hierzu müssen Klienten ihre Nachbarn kennen. Die Adressen der Nachbarn sowie Änderungen der Nachbarschaftsverhältnisse bekommen die Klienten vom Broker mitgeteilt. Gehen Sie dazu wie folgt vor:

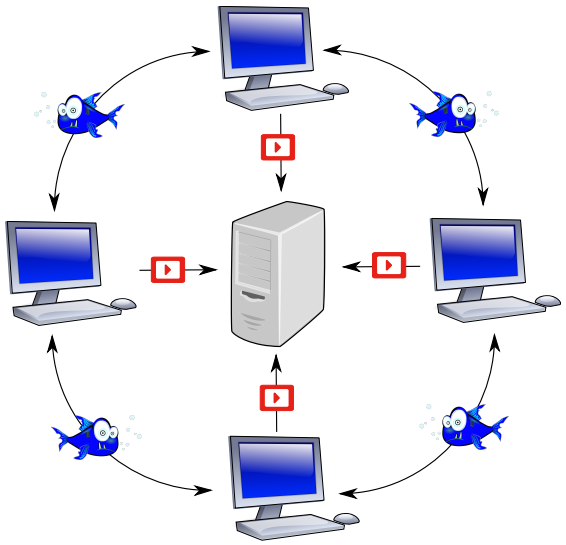


Figure 1: Aqualife als Ring. Klienten registrieren sich beim Broker und erhalten Adressen ihrer Nachbarn. Fische werden direkt zwischen den Klienten versendet.

- Erstellen Sie einen neuen Nachrichtentyp `NeighborUpdate`, mit dem der Broker einem Klienten die `InetSocketAddress` eines neuen linken oder rechten Nachbarn mitteilen kann.
- Erweitern Sie die Methode `register` des Brokers so, dass alle Klienten, die von der Anmeldung des neuen Klienten betroffen sind, die Adressen ihrer linken und rechten Nachbarn gesendet bekommen. Das sind der neue Klient selbst, sowie dessen neuer linker und rechter Nachbar. Beachten Sie, dass der erste Klient in der verteilten Umgebung sein eigener linker und rechter Nachbar ist.
- Erweitern Sie analog die Methode `deregister` des Brokers. Hier sind die betroffenen Klienten der linken und der rechte Nachbar des zu löschenden Klienten.
- Die Klasse `TankModel` muss die `InetSocketAddress`en der Nachbarn halten.
- Erweitern Sie die Klasse `ClientCommunicator.ClientReceiver` so, dass auch `NeighborUpdate`–Nachrichten entgegen genommen und verarbeitet werden.
- Ändern Sie die Klasse Der `ClientCommunicator.ClientForwarder` so, dass beim Hand-Off Fische direkt an die richtigen Nachbarn geschickt werden.

Aufgabe 2: Token-Ring

In dieser Aufgabe sollen sie das Token-Ring-Verfahren zum gegenseitigen Ausschluss implementieren. In Aufgabe 1 haben Sie die Aqualife-Klienten bereits auf eine Ringstruktur umgestellt. Zusätzlich zu den Fischen soll nun ein Token durch den Ring laufen. Klienten dürfen nur dann Fische an Ihre Nachbarn schicken, wenn Sie gerade das Token haben. Ansonsten stoßen sie an die Aquariumsbegrenzung, ändern ihre Richtung und schwimmen zurück. Beachten Sie, dass das Token nur in eine Richtung durch den Ring weitergegeben werden darf, Fische jedoch in beide Richtungen schwimmen können.

- Erstellen Sie den neuen Nachrichtentyp `Token`, der das durch den Ring laufende Token repräsentiert.
- Zum Verwalten des Tokens im `TankModel` benötigen Sie eine boolsche Instanzvariable sowie einen Timer (`java.util.Timer`), dem Sie Aufgaben für die zukünftige Ausführung übergeben können.
- Das `TankModel` muss Methoden zum Empfangen (`receiveToken`) und zum Abfragen des Tokens (`hasToken`) bereitstellen. Empfängt das `TankModel` das Token, wird die boolsche Variable gesetzt und dem `Timer` eine `TimerTask` zur Ausführung nach z.B. 2 Sekunden übergeben. In der `TimerTask` wird die boolsche Variable zurückgesetzt und das Token an den linken Nachbarn weitergeschickt.
- Das `TankModel` darf nur dann Fische an Nachbarn schicken, wenn es gerade das Token hält. Wenn ein Fisch an den Rand des Aquariums stößt, solange das Aquarium das Token nicht hält, ändert es seine Schwimmrichtung. Verwenden Sie dazu die Methode `FishModel.reverse`.

- Die Klassen `ClientForwarder` und `ClientReceiver` müssen so verändert werden, dass sie mit `Token`s umgehen können.
- Der `Broker` gibt das Token an den ersten Client aus, der sich registriert.
- Visualisieren Sie das Wandern des Tokens, indem Sie in der Klasse `TankView` Aquariumsgrenzen zeichnen, solange das Token nicht da ist. Verwenden Sie dazu die Methode `TankView.drawBorders`.

Viel Spass & gutes Gelingen!